

1. Determine whether the following statements are true or false. Write T in the corresponding box if it is true otherwise write F. [10]

1	2	3	4	5	6	7	8	9	10
F	F	T	F	F	T	F	F	T	F

Not in syllabus

- a. The decorator pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable.
- b. HTTP Response Code 200 indicates that connection can't be established.
- ~~c. In the chain of responsibility pattern, the role of the handler objects is to process the request and pass it to the next handler.~~
- d. If a test plan is run for 10 users with a ramp up period 20 seconds, then 2 threads will be started every second.
- e. By default, the git diff command shows differences between the working directory and the remote branch of the repository. *git diff → diff between working directory & staging directory*
- f. Simple factory deals with one factory but factory method pattern and abstract factory can deal with multiple factories.
- g. The git add command directly records changes to the repository by creating a new commit. *git add only moves the change to staging directory (index)*
- h. The git stash command transfers stashed changes to the remote repository when the next git push command is executed. *git stash pop / git stash apply*
- i. **Facade** pattern is used to reduce communication complexity between multiple objects or classes.
- j. The .gitignore file is used to track all uncommitted files in a repository.
↳ doesn't track specified file

2. Define the following Terminologies in brief: [5]

Terminology	Definition
Listener	A part to Jmeter test plan where the receive & respond to a certain event is done
Latency	Time difference between first request sent & first byte read by the servers
Assertion	A condition/rule applied to verify to the respond of our event
Threat	Vulnerability or issues in our event
Attack	A malicious act that does vulnerability event to our program

3. The current repository has two branches, **main** and **dev**. You have mistakenly edited the **main** branch but have not committed the changes. Write the commands to move the changes to the **dev** branch and commit them there. [5]

Ans: `git stash`
`git checkout dev`
`git stash pop`
`git add .`
`git commit -m "dev branch"`

4. For each of the scenarios, identify which pattern it resembles and state the main reason (no more than 10 words): [5]
- A company has two biscuit factories A and B which can produce same biscuits but with different formulas. A is currently in operation but the company might switch to B in the near future.
 - A company has two biscuit factories A and B which produces same kinds of biscuits but one produces sweet version and the other produces savory version.

Ans:

- Strategy Pattern.** Same product but with different formula. We may switch one to another in the future.
- Abstract Factory Pattern.** A family of products is built from different product.

5. Find the protocol, Server Name, Path, parameter name and parameter value from the following web address. [5]

<https://www.google.com/imghp?hl=en&tab=ri>

Ans:

Protocol	https
Server Name	www.google.com
Path	/imghp
Parameter Name	hl, tab
Parameter Value	en, ri

6. The following code checks a student's eligibility for **scholarships** and **TAships**. The `func1` method checks if the student is eligible for a scholarship based on their CGPA. `func2` method checks if the student is eligible for TAship, checking whether they are regular and have cgpa above a threshold. It then sends an email to the student.

Your task is to refactor the code and apply clean coding practices to improve readability and maintainability, as well as adhere to principles such as **meaningful naming conventions, SRP, DRY, etc.** [10]

```

public class StudentSystem {
    public boolean func1(double a) {
        if (a >= 3.5) {
            return true;
        } else {
            return false;
        }
    }

    public void func2(double a, boolean x) {
        if (a >= 3.5) { // Check eligibility by cgpa
            if (x) { // Check whether the student is regular or not
                System.out.println(studentName + " is eligible for TAsip.");
                sendEmail(studentName);
            }
            else {
                System.out.println(studentName + " is not eligible for TAsip.");
            }
        }
        else {
            System.out.println(studentName + " is not eligible for TAsip.");
        }
    }
}

```

Handwritten Annotations:

- Magic numbers:** `3.5` (pointing to `3.5` in `func1` and `func2`)
- Naming convention:** `cgpa` (pointing to `cgpa` in `func2`)
- DRY (Don't Repeat Yourself):** (pointing to the repeated `System.out.println` statements)
- SRP (Single Responsibility Principle):** (pointing to the `func2` method)
- Refactored Code:**

```

public class StudentSystem {
    private final static double cgpa = 3.5;

    public boolean eligible_for_ship(double CGPA) {
        return CGPA > cgpa;
    }

    public void eligible_for_TAsip(double CGPA, boolean isRegular) {
        if (eligible_for_ship(CGPA) && isRegular) {
            notifyStudent();
        }
        else {
            // ...
        }
    }

    void notifyStudent() {
        // ...
        sendEmail();
    }
}

```

Ans:

X Below you are provided with a demo code for a payment processing scenario. Analyze the PaymentProcessor class and determine which **structural design pattern(s)** it follows. Justify your answer by examining the responsibilities and behavior of the class. [10]

```
class PaymentGatewayAPI {
    public void processPayment(String paymentDetails) {
        System.out.println("Processing payment with details: " + paymentDetails);
    }

    public void convertCurrency(String paymentDetails, String targetCurrency) {
        System.out.println("Converting payment to " + targetCurrency + ": " + paymentDetails);
    }

    public String generateTransactionId() {
        System.out.println("Generated Transaction ID: " + transactionId);
        return transactionId;
    }

    public boolean confirmTransaction(String transactionId) {
        System.out.println("Confirming transaction with ID: " + transactionId);
        return true;
    }
}

class PaymentProcessor {
    private PaymentGatewayAPI paymentGateway;

    public PaymentProcessor() {
        paymentGateway = new PaymentGatewayAPI();
    }

    public boolean makePayment(String cardNumber, double amount, String targetCurrency) {
        String formattedDetails = formatPaymentDetails(cardNumber, amount);
        paymentGateway.convertCurrency(formattedDetails, targetCurrency);
        String transactionId = paymentGateway.generateTransactionId();
        paymentGateway.processPayment(formattedDetails);
        return paymentGateway.confirmTransaction(transactionId);
    }

    private String formatPaymentDetails(String cardNumber, double amount) {
        return cardNumber + ":" + amount;
    }
}

public class Main { // Client code
    public static void main(String[] args) {
        PaymentProcessor processor = new PaymentProcessor();

        String userCard = "1234-5678-9876-5432";
        double userAmount = 250.75;
        String targetCurrency = "USD";

        processor.makePayment(userCard, userAmount, targetCurrency);
    }
}
```

Ans:

8. Imagine a music player app that can be either playing music, paused, or stopped. How the app **works** changes depending on what it's doing. For example, if it's playing, you can press pause; if it's paused, you can press play to continue; and if it's stopped, you can press play to start again. → **State**

On top of that, the app lets you choose how you want your music to play. You can pick different modes like shuffle, where the songs play in random order; repeat, where the same song or playlist keeps looping; or normal, where the songs play in the order you picked them. The way the player behaves changes not just with whether it's playing, paused, or stopped but also depending on which playback option you choose, giving you a personalized music experience. → **Strategy**

Write the name(s) of the pattern(s) that you will use to solve the problem. Then, implement the solution. [10]

Ans: interface playbackstrategy {

class normal implements playbackstrategy {

class shuffle " " {

class repeat " " {

interface state {

class Play implements state {

class pause " " {

class stop " " {

class stateDirector {

CSE-214: Software Engineering — Full Practice Exam

Name: Kazi Asef Kabira

Student ID: 2305084

1. Determine whether the following statements are true or false. Write T or F.

#	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	T	F	T	F	F	T	F	T	T	F	F	T	F	T	T

- a. The Builder pattern separates the construction of a complex object from its representation so the same process can create different representations.
- b. The Abstract Factory pattern deals with multiple exactly one factory, while Simple Factory can deal with one factory multiple factories.
- c. The Bridge pattern is used when a class has two independent dimensions that would otherwise cause a subclass explosion.
- d. The Decorator pattern is used to reduce coupling communication complexity between multiple objects or classes.
- e. Template Method is based on composition and lets you change behavior at runtime, while Strategy is based on inheritance.
- f. The Observer pattern establishes a one-to-many, publish/subscribe relationship between a subject and its observers.
- g. In the Mediator pattern, components communicate directly with each other instead of through a central object. → Reverse
- h. A UML dashed line with a hollow (white) arrowhead pointing to the parent represents an interface implementation.
- i. Composition is symbolized by a black (filled) diamond, and the parts live and die with the whole.
- j. By default, git diff compares the working directory with the remote branch of the repository. → staging area
- k. The .gitignore file tracks all uncommitted files so they can later be committed automatically. → doesn't track
- l. SRP states that a class or function should have only one reason to change.
- m. Shotgun Surgery is a code smell where a single class must change for many unrelated reasons. → Divergent change
- n. Latency in JMeter is the time to receive the first byte of the response from the server. Shotgun → multiple
- o. If a test plan runs 20 users with a 40-second ramp-up period, on average one new user thread starts every 2 seconds. class

2. Define the following terminologies in brief: [10]

Terminology	Definition
Bridge	structural pattern where large class is extracted as multiple class to avoid subclass explosion.
Aggregation	Diagram with "has-a" relation.
Association Class	
Detached HEAD	A Git state where Head is pointing at a commit instead of a branch. Occurs when we do <code>git commit [commit hash]</code> , before pointing towards a branch.
Code Smell	Indication of structural errors of the existing codebase.

3. The current repository has two branches, **release** and **hotfix**. You have mistakenly edited the **release** branch and already **committed** the change there (one commit ahead of hotfix). Write the Git commands to move that single commit onto **hotfix** and remove it from **release**. [6]

Ans:

`git checkout hotfix`

`git cherry-pick release` → copies the commit of hotfix as a new commit

`git checkout release`

`git reset --hard HEAD~1` → removes mistaken commit from release

4. For each of the scenarios, identify which pattern it resembles and state the main reason (no more than 10 words): [6]

- ↳ **Decorator pattern.** Any combination on the main product is possible & even after wrapping, it returns the same object
- a. An online store lets a customer add gift-wrapping, express-shipping, and insurance to an order in any combination, and the price should update by wrapping the order object rather than creating a subclass for every combination.
- b. A smart-home app has several sensor widgets (temperature, motion, light) on a dashboard that must all refresh automatically the instant any new sensor reading arrives, and widgets can be added or removed while the app is running.

Ans: ↳ **Observer pattern.** The app is subscribed by the users. Everytime the app gets an update, the users get automatic update of the app

5. Find the protocol, Server Name, Path, parameter name(s) and parameter value(s) from the following web address. [5]

<https://opensource-demo.orangehrmlive.com/web/index.php/auth/login?lang=en&redirect;=1>

Field	Value
Protocol	https
Server Name	opensource-demo.orangehrmlive.com
Path	/web/index.php/auth/login
Parameter Name	lang, redirect
Parameter Value	en, 1

6. The following code checks a customer's eligibility for a **loyalty discount** and for **free shipping**. func1 checks if the customer is eligible for a discount based on their total spend. func2 checks if the customer is eligible for free shipping — checking whether they are a premium member and have spent above a threshold — then sends a confirmation email.

Your task is to refactor the code and apply clean coding practices to improve readability and maintainability, as well as adhere to principles such as **meaningful naming conventions**, **SRP**, **DRY**, etc. [10]

<pre>public class OrderSystem { public boolean func1(double a) { if (a >= 500) { return true; } else { return false; } } public void func2(double a, boolean x) { if (a >= 500) { // check eligibility by spend if (x) { // check whether customer is premium System.out.println(customerName + " is eligible for free shipping."); sendEmail(customerName); } else { System.out.println(customerName + " is not eligible for free shipping."); } } else { System.out.println(customerName + " is not eligible for free shipping."); } } }</pre>	<pre>public class OrderSystem { private final static int threshold_spend = 500; public boolean eligible_for_discount(double total_spend) { return total_spend > threshold_spend; } public void eligible_for_free(double spend, boolean isPremium) { if (eligible_for_discount(spend) & isPremium) { notify(); } else { sout(); } } }</pre>
---	--

Ans:

Naming convention: func1, a, x, func2

SRP: func2

DRY: Duplicated if/else

Single Source of truth: Magic Numbers

void notify() {

sout();

sendEmail();

}

}

7. Below is a demo code for a notification-sending scenario. Analyze the SmsNotifierAdapter class and determine which **structural design pattern** it follows. Justify your answer by examining the responsibilities and behavior of the class. [10]

```
interface Notifier {
    void notify(String recipient, String message);
}

class LegacySmsGateway {
    // third-party class, cannot be modified
    public void sendSms(String phoneNumber, String text) {
        System.out.println("SMS to " + phoneNumber + ": " + text);
    }
}

class SmsNotifierAdapter implements Notifier {
    private LegacySmsGateway gateway;
    public SmsNotifierAdapter(LegacySmsGateway gateway) { this.gateway = gateway; }
    public void notify(String recipient, String message) {
        gateway.sendSms(recipient, message); // translates the call
    }
}

class Main {
    public static void main(String[] args) {
        Notifier notifier = new SmsNotifierAdapter(new LegacySmsGateway());
        notifier.notify("017XXXXXXX", "Your order has shipped");
    }
}
```

Adapters
Wraps the adaptee object

Ans:

Adapters pattern (Object Adapters)

Object Adapters → wraps the adaptee (implements adaptee)

Class adapters → extends a class & implements the interface (extends adaptee implements target)

8. Imagine a food-delivery order that can be in one of these states: Placed, Preparing, OutForDelivery, or Delivered. What actions are allowed (cancel, confirm, mark-delivered) depends on the order's current state — for example, an order can only be cancelled while it is Placed. Separately, the app lets a customer choose how the delivery fee is calculated — Flat Rate, Distance-Based, or Surge — and this calculation can change independently of the order's state.

Strategy

Write the name(s) of the pattern(s) you would use to solve this problem. Then, implement the solution (class/interface sketch is sufficient). [10]

State + strategy

Ans:

interface OrderState {}

class Placed implements OrderState {}

class Prepared implements OrderState {}

interface FeeStrategy {}

class FlatRate implements FeeStrategy {}

class DistanceBased implements FeeStrategy {}

9. A ride-sharing company has multiple Drivers and Riders. Each Driver owns exactly one Vehicle, and a Vehicle cannot exist without its Driver. A Rider can request zero or more Trips; each Trip is associated with exactly one Driver and one Rider, and a Trip records the fare and timestamp for that specific Driver/Rider pairing. We know the driver's name and license number, the rider's name, the vehicle's plate number and model, and the trip's pickup location, drop-off location, fare, and timestamp.

- List the candidate classes. [2]
- Draw (describe in words: classes, attributes, and relationship types with multiplicities) the class diagram for this scenario. [6]
- Identify where an Association Class is needed and why. [2]

Ans:

10. A test plan is run for 60 users with a ramp-up period of 30 seconds.

- On average, how many new user threads start per second? [2] $\rightarrow 60/30 = 2 \text{ users/sec}$
- Write the JMeter CLI command to run the test plan located at /home/tests/Checkout.jmx, log results to /home/tests/results.jtl, and generate an HTML report into /home/tests/report. [4]
- Name three Listeners you could add to view the results of this test. [3] $\rightarrow$ view Results tree, view results in table, Summary Report
- What is the difference between Latency and Sample Time in JMeter? [1]

Ans:

<u>Latency</u> time to receive 1st Byte of the response	<u>Sample Time</u> time to receive complete response
--	--

⑥ `jmeter -n -t /home/tests/Checkout.jmx -l /home/tests/results.jtl -e -o /home/tests/report`

11. Identify the code smell(s) present in the following method, and name the refactoring technique(s) you would apply to fix each one. [6]

```
class ShippingCalculator {  
    public double calculateShippingCost(String customerName, String customerEmail,  
                                       String customerAddress, String customerPhone,  
                                       double weight, double distance) {  
        double cost = weight * 2.5 + distance * 0.1;  
        if (weight > 50) {  
            cost = cost * 1.2; // magic number, repeated in InvoiceService.java too  
        }  
        return cost;  
    }  
}
```

Ans:

1. Long parameters → Parameters Object
2. Magic number → Single Source of Truth
3. Duplicate code → Extract method

12. Short answer questions on Clean Coding: [8]

- a. Why does the lecture describe some comments as "a deodorant masking the smell of fishy code"? Give an example of a comment that should instead become a method name. [4]
- b. Give one example each of a comment that IS welcome, and explain why (one for explaining 'why', one for a complex algorithm). [4]

Ans:

- a. Can't recognize from method but from comments.

End of practice exam. Total marks: 100 (indicative). Good luck!

CSE-214: Git & Version Control — Practice Question Set

Name: Kazi Asef Kabira

Student ID: 2305084

1. Determine whether the following statements are true or false. Write T or F.

#	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	T	F	T	F	T	F	T	T	T	F	T	T	F	F	T

- a. git init creates a new, empty repository by creating a hidden .git folder in the current directory.
- b. Git is a centralized version control system, so only the remote server stores the complete project history. → Local ones also
- c. The Staging Area is a temporary space for preparing exactly which changes will go into the next commit.
- d. git add -A stages only the files that were modified, not newly created or deleted files. → stages all the files
- e. git commit -a stages all currently-tracked, modified files and commits them in one step. → Removes the previous commit message
- f. git commit --amend creates an entirely new commit and keeps the previous one unchanged in the history.
- g. By default, git diff compares your working directory against the most recent local commit, not any remote branch.
- h. git log lists commits in chronological order along with their hashes and messages.
- i. A branch in Git is simply a movable pointer (a named label) that refers to one specific commit.
- j. HEAD always points to a specific commit and can never point to a branch. It only points to a commit in "detached HEAD" state.
- k. A detached HEAD state occurs when you check out a specific commit hash instead of a branch name.
- l. git merge uses the least common ancestor of two branches to determine what changed on each side.
- m. git revert permanently deletes a commit from the project's history. Doesn't delete anything. Creates another commit with opposite changes, keeping history intact
- n. Changes saved with git stash are automatically pushed to the remote repository the next time you run git push.
- o. git cherry-pick applies the changes introduced by one specific commit onto your current branch as a new commit.

2. Define the following terminologies in brief: [8]

Terminology	Definition
Working Directory	Actual file we see & edit on disk.
Commit Hash	Uniquely identifiers for a commit
Cherry-pick	Standard way to move commit one branch to another.
Merge Conflict	A merge between two branches where both branches have changed same line of the same file
.gitignore	A file that lists file that doesn't need to be tracked
Tracked File	Files that git is aware of

3. Match each Git command on the left with its correct description from the options below (write the matching letter next to each command). [8]

Command	Description
git status	check current branch & modified files
git checkout -b feature	Creates & switches to branch "feature"
git stash	Temporarily shelves uncommitted changes so that we can change branch cleanly
git branch -d feature	Delete the branch "feature"
git commit --amend	Remove the previous commit message & write new one.
git diff path/to/file	Difference between last committed & recent written file
git checkout [hash] path/to/file	Copies one specific file
git log	shows past commits

Options (write the matching letter next to each command above): (a) Deletes a branch that has already been merged; (b) Shows which files have been modified/staged and the current branch; (c) Temporarily shelves uncommitted changes so you can switch branches cleanly; (d) Creates a new branch and immediately switches to it; (e) Edits/replaces the most recent commit instead of creating a new one; (f) Copies one specific file, as it existed in a given commit, into the working directory; (g) Shows the lines changed in one specific file since the last commit; (h) Lists every past commit chronologically with its hash and message

4. You are starting a brand-new project. Write the sequence of Git commands to: (a) initialize a repository in the current folder, (b) stage a file named app.py, and (c) make the first commit with the message "Initial commit". [4]

Ans: `git init`
`git add app.py`
`git commit -m "initial commit"`

5. You have made changes to a file named config.yml but have NOT staged or committed them, and you decide you want to throw those changes away and go back to the version from your last commit. Write the single command to do this. [3]

Ans: `git checkout --config.yml`

6. You are on the main branch and want to merge in the completed work from a branch named login-feature. Write out the full merge workflow, step by step, including how you would check for and resolve conflicts. [6]

Ans: `git checkout main`
`git merge --login-feature`
`git status`
`git add .`
`git commit -m "Hello"`

7. While investigating a bug, you ran `git checkout a3f9c21` (a commit hash) and made two new commits while exploring a fix. You now realize you are in a detached HEAD state and want to keep this work permanently. What command do you run, and why does this situation happen in the first place? [5]

Ans: `git checkout -b branch_name`

8. The current repository has two branches, **staging** and **production**. You have mistakenly committed a change directly on **production** (one commit ahead of staging). Write the Git commands to copy that commit onto **staging** and then remove it from **production**. [6]

Ans: `git checkout staging`
`git cherry-pick production`
`git checkout production`
`git reset --hard HEAD~1`

9. You are halfway through editing a file on branch feature-x when you're asked to quickly fix an urgent bug on main. You have not committed your feature-x changes yet. Describe the sequence of commands you would use to switch to main, make and commit the urgent fix, and then return to feature-x with your original uncommitted changes restored. [6]

Ans: `git stash`
`git checkout main`
`git add .`
`git commit -m "Fix"`
`git stash pop`

10. Find the protocol, Server Name, Path, parameter name(s) and parameter value(s) from the following web address. [5]

<https://github.com/search?q=design+patterns&type=repositories>

Field	Value
Protocol	https
Server Name	github.com
Path	/search
Parameter Name	q, type
Parameter Value	design+pattern, repositories

11. Short answer questions: [8]

diff between lines
↑
→ past commits

a. Explain the difference between git diff and git log — what does each one actually show you? [4]

b. Explain why git revert is generally considered safer than deleting or rewriting a commit that has already been pushed and shared with a team. [4] → never deletes anything, add new commit branch & does the

Ans:

opposite branch

12. A classmate ran the following sequence of commands and is confused about why their teammate can't see their stashed changes after pulling the latest code. Identify what's wrong with their understanding, and explain what they should have done instead. [6]

```
$ git checkout feature-x
$ git stash —→ takes new commit
$ git push origin feature-x
# ... tells teammate: "I stashed my changes and pushed, you should have them now"
```

Ans:

→ Didn't get new commit

End of Git practice question set. Good luck!

CSE-214: Code Refactoring — Practice Question Set

Name: Kazi Asef Kabin

Student ID: 2305084

1. Determine whether the following statements are true or false. Write T or F.

#	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	T	F	F	F	F	T	F	F	T	T	F	T	T	T	T

- a. Refactoring changes the internal structure of code without changing its external, observable behavior.
- b. Performance optimization and refactoring both tend to improve code readability in the same way.
- c. The Rule of Three suggests you should refactor the first time you write a piece of code.
- d. Refactoring should only be done at the very end of a project, right before release.
- e. A code smell is a bug that causes the program to produce incorrect output.
- f. Duplicated Code is risky because a bug fix applied to one copy may be missed in the others.
- g. Divergent Change means a single kind of change forces you to edit many different classes.
- h. Shotgun Surgery means a single class must be edited for many different, unrelated reasons.
- i. Feature Envy is a sign that a method may be more interested in another class's data than its own.
- j. A Long Parameter List is typically fixed using the Introduce Parameter Object refactoring.
- k. Primitive Obsession refers to overusing small, purpose-built classes instead of raw primitive types.
- l. Replace Conditional with Polymorphism is commonly used to eliminate a switch statement that checks an object's type.
- m. A Message Chain occurs when a client calls a.getB().getC().getD(), depending on the entire navigation path.
- n. Refused Bequest is a sign that a subclass may not actually pass the IS-A substitutability test.
- o. Guard Clauses are used specifically for conditionals that check unusual, special-case circumstances before falling through to one normal path.

2. Define the following terminologies in brief: [8]

Terminology	Definition
Code Smell	Code works but internally we have structural code issues
Divergent Change	Irrelevant issues that made the class change the code
Data Clumps	same group of attributes keeps coming together
Lazy Class	Due to refactoring it is downsized
Middle Man	A class that delegates more than half of the responsibility to other class
Guard Clause	An early return statement placed on top of that method

3. Match each code smell on the left with its most appropriate refactoring fix from the options below (write the matching letter next to each smell). [8]

Code Smell	Recommended Fix
Duplicated Code	c
Long Method	f
Feature Envy	a
Long Parameter List	b
Message Chain	e
Switch Statement on type	d

Options (write the matching letter next to each smell above): (a) Move Method (relocate the method to the class whose data it uses most); (b) Introduce Parameter Object (bundle the related parameters into their own class); (c) Extract Method (pull the shared fragment into its own well-named method); (d) Replace Conditional with Polymorphism (let each subclass override its own behavior); (e) Hide Delegate (let the intermediate class expose a method that forwards the call); (f) Extract Method, repeatedly, to break the long method into smaller, well-named pieces

4. For each description, state whether it is Divergent Change or Shotgun Surgery: [4]

a. Every time the tax law changes, the UI display format changes, or the email template changes, you find yourself editing the same Employee class. **Divergent**

b. Changing a single minimum-order business rule requires you to edit a validation class, a pricing class, and a notifications class, all at once. **Shotgun**

Ans:

5. Identify the code smell(s) present in the following method, and name the refactoring technique(s) you would apply. [8]

```
class DiscountCalculator {  
    public double getDiscount(String customerType) {  
        if (customerType.equals("REGULAR")) {  
            return 0.05;  
        } else if (customerType.equals("SILVER")) {  
            return 0.10;  
        } else if (customerType.equals("GOLD")) {  
            return 0.15;  
        } else if (customerType.equals("PLATINUM")) {  
            return 0.20;  
        }  
        return 0.0;  
    }  
}
```

* Duplicate Code
* Switch State

```
// The SAME if/else chain on customerType also appears, separately,  
// inside ShippingCalculator.getFreeShippingThreshold(customerType)
```

Ans:

6. Refactor the following method using Extract Variable and/or Extract Method to make it more readable, without changing what it computes. [8]

```
class InvoiceCalculator {  
    public double getFinalAmount(double qty, double unitPrice, double taxRate) {  
        return (qty * unitPrice) - ((qty * unitPrice) * (qty > 100 ? 0.15 : 0.05))  
            + ((qty * unitPrice) * taxRate);  
    }  
}
```

each one gets variable name

Ans:

7. Refactor the following method using Guard Clauses. [8]

```
double getShippingCost(Order order) {  
    double cost;  
    if (order.isCancelled()) {  
        cost = 0;  
    } else {  
        if (order.isInternational()) {  
            cost = 25.0;  
        } else {  
            if (order.isExpress()) {  
                cost = 15.0;  
            } else {  
                cost = 5.0;  
            }  
        }  
    }  
    return cost;  
}
```

Ans:

8. Identify the code smell in the following class, and refactor using Move Method. [8]

```
class Rectangle {  
    private double width, height;  
    public double getWidth() { return width; }  
    public double getHeight() { return height; }  
}  
class AreaReportPrinter {  
    public void printArea(Rectangle r) {  
        double area = r.getWidth() * r.getHeight();    // uses ONLY Rectangle's data  
        System.out.println("Area: " + area);  
    }  
}
```

** Feature Envy*

Ans:

9. The following method signature repeats the same three parameters in several other methods of the same class (not shown). Identify the smell and refactor using Introduce Parameter Object. [6]

```
class GeoService {  
    public double distanceFromOffice(double latitude, double longitude, double altitude) { ... }  
    public boolean isWithinCity(double latitude, double longitude, double altitude, String city)  
    { ... }  
}
```

Ans:

Data clumps

10. The following class exposes its internal collection directly. Identify the risk this creates, and refactor using Encapsulate Collection. [6]

```
class Classroom {  
    private List students = new ArrayList<>();  
    public List getStudents() { return students; } → .immutableList ( ):  
    public void setStudents(List students) { this.students = students; } → instead add/remove  
}  
// Elsewhere in the codebase:  
classroom.getStudents().clear(); // wipes ALL students with no validation or warning
```

Ans:

11. Short answer questions: [8]

a. Explain the difference between refactoring and performance optimization, in terms of what each preserves and what each typically does to readability. [4]

b. Explain the Rule of Three, and give a short example of when you would apply it. [4]

Ans:

1. First write only
2. Find duplicates
3. Remove duplicates

12. The following class has MULTIPLE code smells. List as many as you can find, and briefly state the fix for each. [12]

```
class OrderManager {
    public void process(String custName, String custEmail, String custPhone,
                        String custStreet, String custCity, String custZip,
                        double amount, String status) {
        if (status.equals("NEW")) {
            // validate address
            if (custStreet == null || custCity == null || custZip == null) {
                throw new IllegalArgumentException("Bad address");
            }
            double fee = amount * 0.03; // processing fee, also hard-coded in BillingService.
            System.out.println("Processing new order for " + custName);
        } else if (status.equals("CANCELLED")) {
            System.out.println("Order cancelled for " + custName);
        } else if (status.equals("SHIPPED")) {
            System.out.println("Order shipped for " + custName);
        } else if (status.equals("REFUNDED")) {
            System.out.println("Order refunded for " + custName);
        }
    }
}
```

Long method

→ Long parameters

→ magic number

→ Switch statements on type

Ans:

End of Refactoring practice question set. Good luck!